Code-to-Hardware Guide

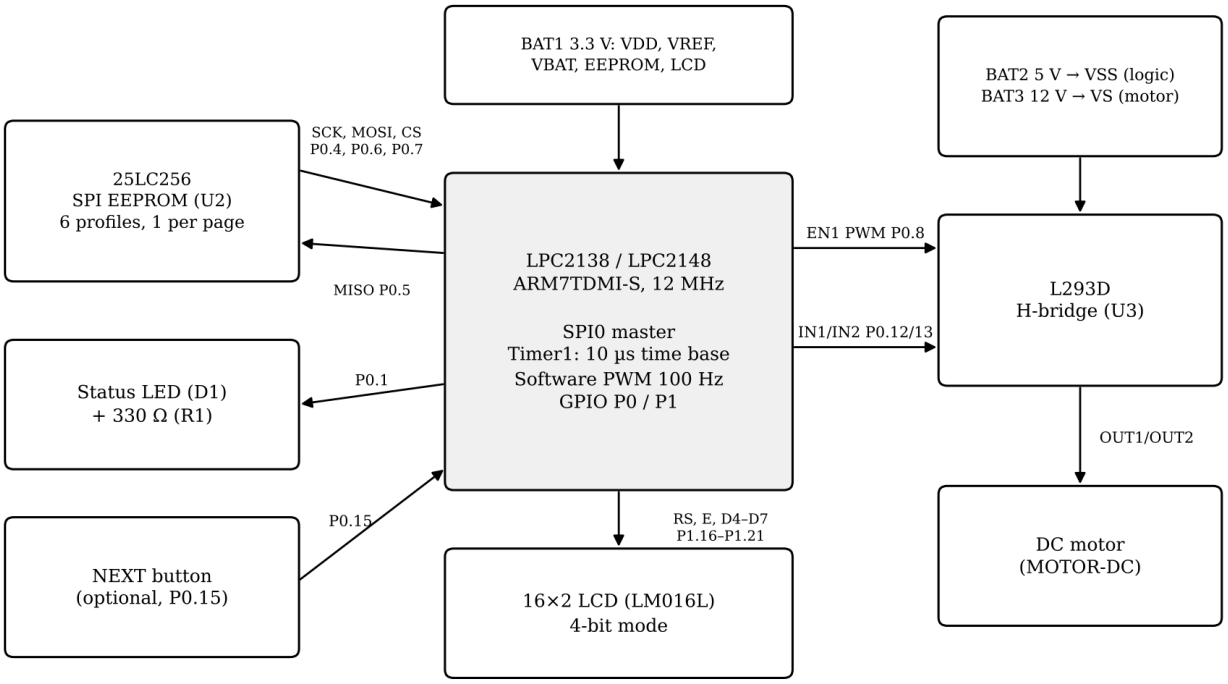
SPI EEPROM Motor-Profile Controller — what every line of `main.c` does to the hardware

> ***How to use this guide.*** *The code (299 lines, the version in the report) is shown in tables. For each line you see* ***the code*** *and* ***what physically happens****: which pin changes, to what voltage, and what the EEPROM, LCD, LED or motor does as a result. Lines that only work on data in memory are marked as such, so you can tell “software” lines from “hardware” lines.*

Team 1, Department of ECE, MNNIT Allahabad. Microcontroller: LPC2138 (same code runs on LPC2148).

---

1. The hardware the code controls

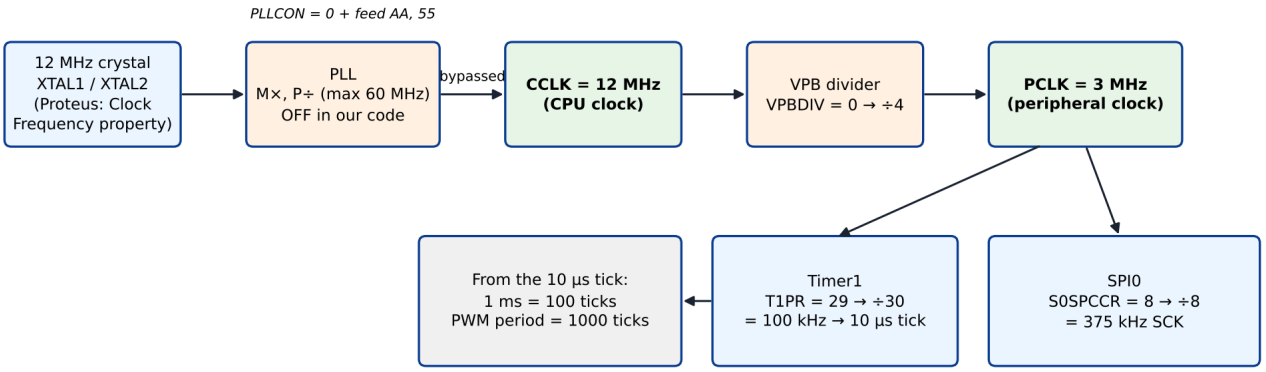


1.1 Every wire the program uses

LPC2138 pin (number)	Name in code	Connected to	3.3 V means	0 V means
P0.1 (21)	`LED`	330 Ω → LED → GND	LED on (≈ 4.5 mA)	LED off
P0.4 (27)	SCK (SPI)	EEPROM SCK (pin 6)	Clock pulses, made by the SPI block	— (idle)
P0.5 (29)	MISO (SPI)	EEPROM SO (pin 2)	Data bit 1 from the EEPROM	Data bit 0
P0.6 (30)	MOSI (SPI)	EEPROM SI (pin 5)	Data bit 1 to the EEPROM	Data bit 0
P0.7 (31)	`CS`	EEPROM CS (pin 1)	EEPROM ignores the bus	EEPROM listens
P0.8 (33)	`EN1`	L293D EN1 (pin 1)	Driver outputs on (motor powered)	Outputs off (motor coasts)
P0.12 (38)	`IN1`	L293D IN1 (pin 2)	Output 1 → 12 V (forward)	Output 1 → 0 V
P0.13 (39)	`IN2`	L293D IN2 (pin 7)	Output 2 → 12 V (reverse)	Output 2 → 0 V
P0.15 (45)	button	Push-button to GND, 10 kΩ to 3.3 V	Released	Pressed
P1.16 (16)	RS	LCD RS (pin 4)	Byte is a character	Byte is a command
P1.17 (12)	E	LCD E (pin 6)	—	Falling edge = LCD reads data
P1.18–P1.21	D4–D7	LCD D4–D7 (pins 11–14)	Data bits	Data bits

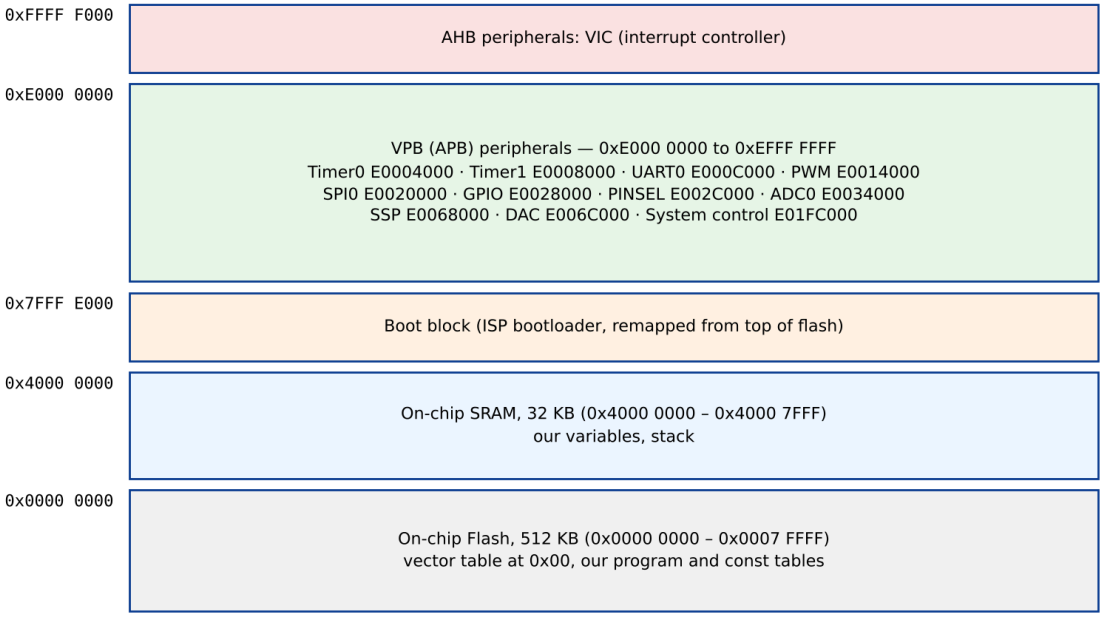
1.2 How the chip's clocks are set

LPC2138 clock tree as used in the project



1.3 Where things live in the chip

LPC2138 memory map (4 GB address space)



Three kinds of lines in the code:

1. **Register lines** — write or read a hardware register (`IO0SET = …` , `S0SPDR = …` , `T1TC`). These change pins or peripherals **immediately**.
2. **Lines that call register functions** — e.g. `pin(LED, 1)` , `spi(0x05)` , `lcd_byte(…)` . They change the hardware through those functions.
3. **Memory-only lines** — work on variables in RAM (`lcd[]` , `buf[]` , `now` , …). They decide *what* the hardware will do next, but no pin moves on that line.

2. Line-by-line: code → hardware

Header comment (lines 1–3)

Line	Code → what happens in the hardware

1–3	`/* SPI EEPROM Motor-Profile Controller - LPC2138 / LPC2148`
	`EEPROM 25LC256: SCK P0.4, MISO P0.5, MOSI P0.6, CS P0.7 | L293D: EN1 P0.8, IN1 P0.12, IN2 P0.13`
	`LED P0.1 | NEXT button P0.15 | LCD: RS P1.16, E P1.17, D4-D7 P1.18-P1.21 */`
	A comment — the compiler ignores it, **nothing happens in the hardware**. It records the wiring: which LPC2138 pin goes to which pin of the EEPROM, L293D, LED, button and LCD.

Register addresses (lines 4–24)

Inside the LPC2138, every peripheral is controlled by **registers** — small 32-bit storage cells wired directly to the hardware. Each register has a fixed address. These lines give each address a name. **No hardware changes when these lines are compiled**; the hardware changes later, when the program *writes* to or *reads* from these names.

Line	Code → what happens in the hardware
4	`#define PINSEL0 (*(volatile unsigned int *)0xE002C000)` `PINSEL0` (address 0xE002C000) controls the **pin multiplexers** of pins P0.0–P0.15. Writing it decides whether each pin is connected to the GPIO logic or to a peripheral such as SPI.
5	`#define PINSEL2 (*(volatile unsigned int *)0xE002C014)` `PINSEL2` decides whether port-1 pins P1.16–P1.25 are connected to the chip's **trace (debug) port** or work as normal GPIO.
6	`#define IO0PIN  (*(volatile unsigned int *)0xE0028000)` `IO0PIN`: **reading** it gives the real voltage level on every port-0 pin (1 ≈ above 2 V, 0 ≈ near 0 V). Used for the button.
7	`#define IO0SET  (*(volatile unsigned int *)0xE0028004)` `IO0SET`: writing a 1 in a bit makes that **output pin go to 3.3 V**. Bits written as 0 are not touched.
8	`#define IO0DIR  (*(volatile unsigned int *)0xE0028008)` `IO0DIR`: a 1 in a bit **switches on the output driver** of that pin (output); 0 = input.
9	`#define IO0CLR  (*(volatile unsigned int *)0xE002800C)` `IO0CLR`: writing a 1 in a bit makes that **output pin go to 0 V**.

10–12	`#define IO1SET  (*(volatile unsigned int *)0xE0028014)`
	`#define IO1DIR  (*(volatile unsigned int *)0xE0028018)`
	`#define IO1CLR  (*(volatile unsigned int *)0xE002801C)`
	The same SET / DIR / CLR registers for **port 1** — they drive the LCD pins P1.16–P1.21.

13	`#define S0SPCR  (*(volatile unsigned int *)0xE0020000)` `S0SPCR`: SPI control — master or slave, clock polarity and phase.
14	`#define S0SPSR  (*(volatile unsigned int *)0xE0020004)` `S0SPSR`: SPI status — the SPI hardware sets bit 7 (**SPIF**) when a byte has finished, and bits 3/4 on errors.
15	`#define S0SPDR  (*(volatile unsigned int *)0xE0020008)` `S0SPDR`: SPI data — **writing it makes the SPI hardware send 8 bits out on MOSI and 8 clock pulses on SCK**; reading it gives the byte received on MISO.

Line	Code → what happens in the hardware
16	`#define S0SPCCR (*(volatile unsigned int *)0xE002000C)` `S0SPCCR`: SPI clock divider — sets the **frequency of the SCK pin**.
17	`#define T1TCR  (*(volatile unsigned int *)0xE0008004)` `T1TCR`: Timer1 control — starts, stops or resets the Timer1 counter circuit.
18	`#define T1TC  (*(volatile unsigned int *)0xE0008008)` `T1TC`: Timer1's **32-bit counter** itself — the hardware adds 1 to it at a fixed rate.
19	`#define T1PR  (*(volatile unsigned int *)0xE000800C)` `T1PR`: Timer1 prescaler — how many peripheral-clock cycles go by before the counter adds 1.

20–21	`#define PLLCON  (*(volatile unsigned int *)0xE01FC080)`
	`#define PLLFEED (*(volatile unsigned int *)0xE01FC08C)`
	`PLLCON` / `PLLFEED`: control of the **PLL** (the circuit that can multiply the crystal frequency up to 60 MHz) and its "unlock" register.

22	`#define PCONP  (*(volatile unsigned int *)0xE01FC0C4)` `PCONP`: **power/clock switches** for each peripheral; a 0 bit stops that peripheral's clock to save power.
23	`#define VPBDIV  (*(volatile unsigned int *)0xE01FC100)` `VPBDIV`: divider between the CPU clock and the **peripheral clock** (PCLK).
24	`#define SCS  (*(volatile unsigned int *)0xE01FC1A0)` `SCS`: chooses whether ports 0/1 are driven by these legacy registers or by the newer "fast GPIO" registers.

How `(*(volatile unsigned int *)0xE0028004)` works: the number is an address; `(volatile unsigned int *)` says "this address is a 32-bit hardware register"; the `*` in front reads or writes it. `volatile` forces the processor to really access the hardware every time — without it the compiler could skip reads of the timer or SPI status and the program would hang.

Settings and pin masks (lines 26–39)

These are names for numbers. They take **no memory and cause no hardware action** by themselves — they make the later lines readable.

Line	Code → what happens in the hardware
26	`#define FOSC 12000000` `FOSC` = 12 000 000: the **crystal frequency** (in Proteus, U1's *Clock Frequency* property). All timing is calculated from it, so it must match the real clock.
27	`#define MS  100  /* Timer1 counts every 10 us, so 100 counts = 1 ms */` `MS` = 100: Timer1 will count every 10 µs, so 100 counts = **1 ms**.
28	`#define LED  (1 << 1)` `LED` = bit 1 → **P0.1 (pin 21)** → 330 Ω resistor → LED → ground.
29	`#define CS  (1 << 7)` `CS` = bit 7 → **P0.7 (pin 31)** → EEPROM chip-select (25LC256 pin 1).
30	`#define EN1  (1 << 8)` `EN1` = bit 8 → **P0.8 (pin 33)** → L293D enable 1 (pin 1). High = driver outputs on.

Line	Code → what happens in the hardware

31–32	`#define IN1  (1 << 12)`
	`#define IN2  (1 << 13)`
	`IN1` = bit 12 → **P0.12 (pin 38)** → L293D input 1 (pin 2); `IN2` = bit 13 → **P0.13 (pin 39)** → L293D input 2 (pin 7). They choose the direction.

33–35	`#define STOP 0  /* directions */`
	`#define FWD  1`
	`#define REV  2`
	Direction codes: STOP 0, FWD 1, REV 2. These numbers are also stored in the EEPROM.

36–39	`#define DECEL 1  /* phases (0 = idle) */`
	`#define DEAD  2`
	`#define ACCEL 3`
	`#define HOLD  4`
	Phase codes (0 = idle): DECEL 1, DEAD 2, ACCEL 3, HOLD 4. They decide the **LCD text** and the **LED pattern**.

Profile tables (lines 41–59)

`const` tables are stored in the chip's **flash memory** (they cannot change and use no RAM).

Line	Code → what happens in the hardware
41	`/* Default profiles: for each step direction, speed %, ramp ms, run ms */` Comment: what the four tables mean.

42–44	`const char def_name[6][7]  = { "DEMO", "WASH", "SPIN", "GATE", "FAN", "MIXER" };`
	`const unsigned int def_reps[6]  = { 1, 3, 1, 1, 2, 3 };`
	`const unsigned int def_steps[6] = { 4, 4, 3, 4, 4, 2 };`
	Names, repeat counts and step counts of the six default profiles.

45–56	`const unsigned int def_dir[6][4] = {`
	`  {FWD, STOP, REV, STOP}, {FWD, STOP, REV, STOP}, {FWD, FWD, STOP, 0},`
	`  {FWD, STOP, REV, STOP}, {FWD, FWD, FWD, FWD}, {FWD, STOP, 0, 0} };`
	`const unsigned int def_speed[6][4] = {`
	`  {75, 0, 40, 0}, {60, 0, 60, 0}, {40, 100, 0, 0},`
	`  {80, 0, 80, 0}, {30, 60, 90, 50}, {100, 0, 0, 0} };`
	`const unsigned int def_ramp[6][4] = {`
	`  {500, 300, 500, 300}, {300, 300, 300, 300}, {1000, 2000, 3000, 0},`
	`  {800, 800, 800, 800}, {1000, 1000, 1000, 1000}, {200, 200, 0, 0} };`
	`const unsigned int def_run[6][4] = {`
	`  {3000, 1000, 3000, 1000}, {2000, 500, 2000, 500}, {2000, 5000, 2000, 0},`
	`  {4000, 3000, 4000, 2000}, {2000, 2000, 2000, 2000}, {700, 500, 0, 0} };`
	Direction, speed %, ramp time and run time of every step, one row per profile. These bytes are **sent to the EEPROM** only when a page there is empty or damaged.

57–58	`const char dir_text[3][5]  = { "STOP", "FWD", "REV" };`
	`const char phase_text[5][6]  = { "IDLE", "DECEL", "DEAD", "ACCEL", "HOLD" };`
	Words shown on the LCD for each direction and phase.

59	`const unsigned int pattern[4] = { 0x001, 0x01F, 0x005, 0x155 };  /* LED: stop, fwd, rev, ramp */` LED blink patterns. Each is 10 bits = ten 100 ms slots of one second: 0x001 one flash (stop), 0x01F five slots on, five off (forward), 0x005 double flash (reverse), 0x155 every other slot (ramping).

Variables in RAM (lines 60–66)

These variables live in the chip's **SRAM** (from address 0x4000 0000). They are set to 0 before `main()` starts.

Line	Code → what happens in the hardware

60–61	`char name[6][7];  /* profiles loaded from the EEPROM */`
	`unsigned int reps[6], nsteps[6], dir[6][8], speed[6][8], ramp[6][8], run[6][8];`
	The profiles **copied from the EEPROM** into RAM: name, repeats, steps and the four step tables (up to 8 steps each).

62	`unsigned char buf[64];  /* one EEPROM page */` `buf[64]` : a RAM copy of **one 64-byte EEPROM page** — filled by an SPI read, or prepared before an SPI write.
63	`char lcd[32];  /* LCD text, 2 lines x 16 */` `lcd[32]` : the 32 characters that **should** be on the LCD (line 1 = 0–15, line 2 = 16–31). The LCD glass only changes when these bytes are sent over the LCD pins.

64–66	`unsigned int now, duty, pwm_start, ms_mark, last_dir, off_since, running, skip, btn, btn_count;`
	`unsigned int led_ms, led_slot, led_pat, lcd_left, lcd_time, spi_err;`
	`unsigned int profile, rep, step, cur_dir, phase, fail, cycles;`
	Timing and state: `now` (planned time of the next event), `duty` (EN1 on-time %), `pwm_start` , `ms_mark` , direction memory for the dead time, button debounce, LED and LCD counters, SPI error, and what is running (`profile` , `rep` , `step` , `cur_dir` , `phase`).

Two helpers (lines 67–68)

Line	Code → what happens in the hardware
67	`void pin(unsigned int mask, int on) { if (on) IO0SET = mask; else IO0CLR = mask; }` `pin(mask, on)` : writes the mask to `IO0SET` or `IO0CLR` . **Result: the selected port-0 pin goes to 3.3 V (on) or 0 V (off)** within one bus cycle. Used for EN1 and the LED.
68	`int reached(unsigned int t) { return (int)(T1TC - t) >= 0; }  /* safe across timer overflow */` `reached(t)` : **reads the Timer1 counter** and answers “has time t come?”. No pin changes. The subtraction trick keeps it correct when the 32-bit counter wraps to 0 after about 11.9 hours.

LCD driver (lines 70–83)

The LCD’s HD44780 controller reads its pins when the **E** pin falls from high to low. In 4-bit mode, one byte needs two such pulses.

Line	Code → what happens in the hardware

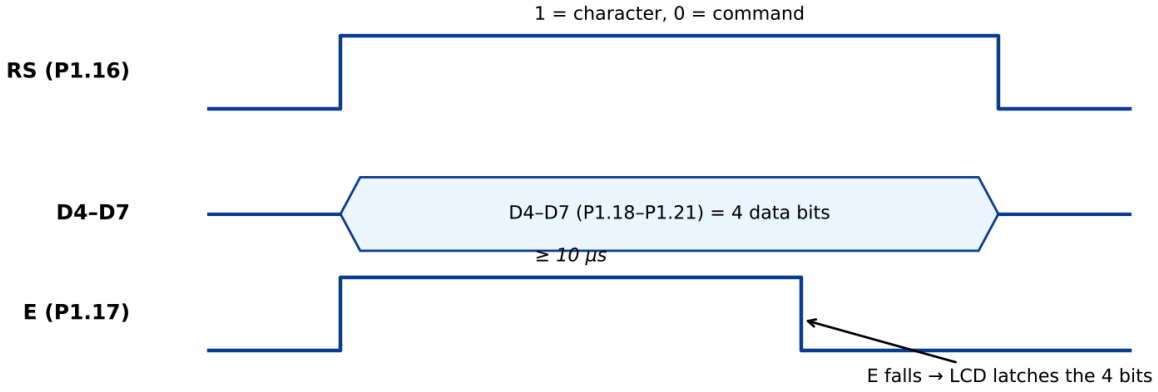
70–71	`/* ---------- LCD (4-bit) ---------- */`
	`void lcd_nibble(int rs, int v) {`
	Section comment and start of `lcd_nibble(rs, v)` — sends 4 bits to the LCD.

72	`    unsigned int t;` A local variable (kept in a CPU register).
73	`    IO1CLR = 0x3F << 16;` **P1.16–P1.21 all go to 0 V** — RS, E and D4–D7 are cleared.
74	`    IO1SET = (rs << 16) | ((v & 15) << 18) | (1 << 17);  /* data, E high */` In one write: **RS (P1.16 → LCD pin 4)** = `rs` ; the 4 bits of `v` appear on **D4–D7 (P1.18–P1.21 → LCD pins 11–14)**; **E (P1.17 → LCD pin 6) rises to 3.3 V**.
75	`    t = T1TC; while (T1TC - t < 2);  /* keep E high */` Read the timer and wait until it has advanced 2 counts (10–20 µs). **E stays high and the data stays stable** — the LCD needs at least 0.45 µs.

Line	Code → what happens in the hardware
76	`    IO1CLR = 1 << 17;                                  /* E low: LCD reads */` **E falls to 0 V → the LCD latches the 4 bits** on D4–D7 into its controller.
77	`}` End of function.
78	`void lcd_byte(int rs, int v) { lcd_nibble(rs, v >> 4); lcd_nibble(rs, v); }` `lcd_byte` : **two E pulses** — upper 4 bits, then lower 4 bits. With RS = 0 the LCD treats the byte as a **command** (e.g. clear, move cursor); with RS = 1 it **prints the character**.

79–80	`void lcd_clear(void) { int i; for (i = 0; i < 32; i++) lcd[i] = ' '; }`
	`void lcd_text(int pos, const char s[]) { int i; for (i = 0; s[i] != 0; i++) lcd[pos + i] = s[i]; }`
	`lcd_clear` / `lcd_text` : change only the RAM array `lcd[]` . **No LCD pin moves yet.**

81–83	`void lcd_num(int pos, int w, unsigned int v) {            /* right-aligned number */`
	`    do { w--; lcd[pos + w] = '0' + v % 10; v = v / 10; } while (v > 0 && w > 0);`
	`}`
	`lcd_num` : writes the digits of a number into `lcd[]` (RAM only). `'0' + v % 10` turns a digit into its ASCII character code.



Background jobs — `service()` (lines 85–109)

`service()` is called again and again inside every wait. Each call does a tiny piece of every background job, so the pins keep doing the right thing while the program waits.

Line	Code → what happens in the hardware

85–86	`/* ---------- Background jobs, run inside every wait ---------- */`
	`void service(void) {`
	Section comment and start of `service()` .

87	`    unsigned int t = T1TC, p;` **Read Timer1** once; `t` is “now” for this whole call.

88–89	`    while (t - ms_mark >= MS) {                           /* every 1 ms */`
	`        ms_mark += MS;`
	For every millisecond that has passed (usually 0 or 1): move the millisecond mark.

90	`        if (++led_ms == 100) { led_ms = 0; if (++led_slot == 10) led_slot = 0; }` LED timing counters in RAM: a new 100 ms slot every 100 ms, ten slots per second.
91	`        p = (IO0PIN >> 15) & 1;                           /* button (0 = pressed), 20 ms debounce */` **Read pin P0.15 (pin 45)**: 1 when the button is released (10 kΩ pull-up holds it at 3.3 V), 0 when pressed (button connects it to ground).

Line	Code → what happens in the hardware

92–94	`if (running == 0 || p == btn) btn_count = 0;`
	`else if (++btn_count >= 20) { btn = p; btn_count = 0; if (p == 0) skip = 1; }`
	`}`
	Debounce in RAM: the new level must stay the same for **20 ms** in a row (contacts bounce for a few ms). A real press sets `skip = 1`.

95	`while (t - pwm_start >= 10 * MS) pwm_start += 10 * MS;` Keep `pwm_start` at the beginning of the current **10 ms PWM period**.
96	`pin(EN1, t - pwm_start < duty * 10);  /* software PWM, 10 ms period */` **Software PWM on P0.8 → L293D EN1.** EN1 is 3.3 V for the first `duty` % of every 10 ms and 0 V for the rest. While EN1 is high, the L293D connects the **12 V motor supply** to the motor; so the motor gets an average voltage of about duty × 12 V (minus the driver's drop). 75 % → on 7.5 ms, off 2.5 ms.

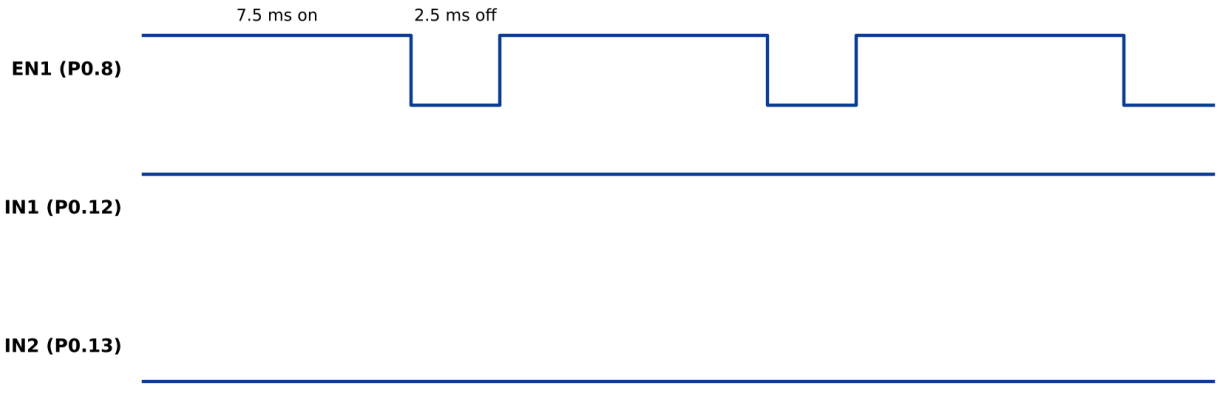
97–99	`if (running) {  /* LED pattern */`
	`p = (phase == ACCEL || phase == DECEL) ? 3 : cur_dir;`
	`if (p + 1 != led_pat) { led_pat = p + 1; led_ms = 0; led_slot = 0; }`
	Choose the LED pattern: ramping → pattern 3, otherwise the direction (0 stop, 1 forward, 2 reverse). A new pattern restarts at slot 0.

100	`pin(LED, (pattern[p] >> led_slot) & 1);` **P0.1 goes to 3.3 V or 0 V** according to the pattern → current flows through the 330 Ω resistor and the **LED lights** (≈ 4.5 mA) or goes dark.
101	`}` End of the LED part.
102	`if (lcd_left > 0 && t - lcd_time >= 5) {  /* one LCD byte every 50 us */` If LCD bytes are waiting and 50 µs have passed since the last one…
103	`p = 34 - lcd_left--;` …work out which of the 34 bytes is next (2 cursor commands + 32 characters).

104–105	`if (p == 0) lcd_byte(0, 0x80);`
	`else if (p == 17) lcd_byte(0, 0xC0);`
	Byte 0: command **0x80** → the LCD moves its cursor to **line 1, column 0**. Byte 17: command **0xC0** → **line 2, column 0**.

106	`else lcd_byte(1, lcd[p < 17 ? p - 1 : p - 2]);` Any other byte: RS = 1 → the LCD **prints that character** and moves its cursor right.

107–109	`lcd_time = T1TC;`
	`}`
	`}`
	Remember when this byte was sent; end of `service()`.



10 ms period (100 Hz) · duty 75 % → motor gets 12 V for 75 % of the time → average ≈ 0.75 × 12 V (minus L293D drop)

Waiting and the LCD status screen (lines 110–123)

Line	Code → what happens in the hardware

110–113	`int wait_until(unsigned int t) {  /* 0 if NEXT was pressed */`
	`    while (!reached(t)) { if (skip) return 0; service(); }`
	`    return 1;`
	`}`
	`wait_until(t)` : keeps calling `service()` until **Timer1 reaches t**. Meanwhile EN1, the LED and the LCD keep working. Returns 0 at once if NEXT was pressed.

114	`void delay_ms(unsigned int ms) { unsigned int t = T1TC + ms * MS; while (!reached(t)) service(); }` `delay_ms(ms)` : a plain wait “from now” (start-up only); `service()` keeps running inside it.

115–116	`void show_status(unsigned int target) {  /* e.g. "WASH FWD 60%" */`
	`    lcd_clear();`
	`show_status(target)` : builds the new screen in RAM, starting from spaces.

117–118	`    lcd_text(0, name[profile]); lcd_text(7, dir_text[cur_dir]);`
	`    lcd_num(11, 3, target); lcd_text(14, "%");`
	Line 1: profile name at column 0, direction at 7, target speed in 11–13, ‘%’ at 14.

119	`    lcd_text(16, (cur_dir == STOP && phase == HOLD) ? "OFF" : phase_text[phase]);` Line 2: the phase word (ACCEL, HOLD, DECEL, DEAD) or “OFF” when stopped.

120–121	`    lcd_text(22, "S /   R /");`
	`    lcd_num(23, 1, step + 1); lcd_num(25, 1, nsteps[profile]);`
	Step number and number of steps, e.g. `S1/4`.

122	`    lcd_num(29, 1, rep + 1);  lcd_num(31, 1, reps[profile]); lcd_left = 34;` Repeat number, e.g. `R2/3`. Then `lcd_left = 34` — **from now on `service()` sends the 34 bytes to the LCD pins**, one every 50 µs; the glass shows the new text about 2 ms later.
123	`}` End of function.

SPI byte transfer (lines 125–136)

Line	Code → what happens in the hardware

125–127	`/* ---------- SPI and 25LC256 EEPROM ---------- */`
	`unsigned char spi(unsigned char tx) {  /* send and receive one byte */`
	`    unsigned int timeout = 100000, s;`
	Section comment; start of `spi(tx)` ; a time-out counter.

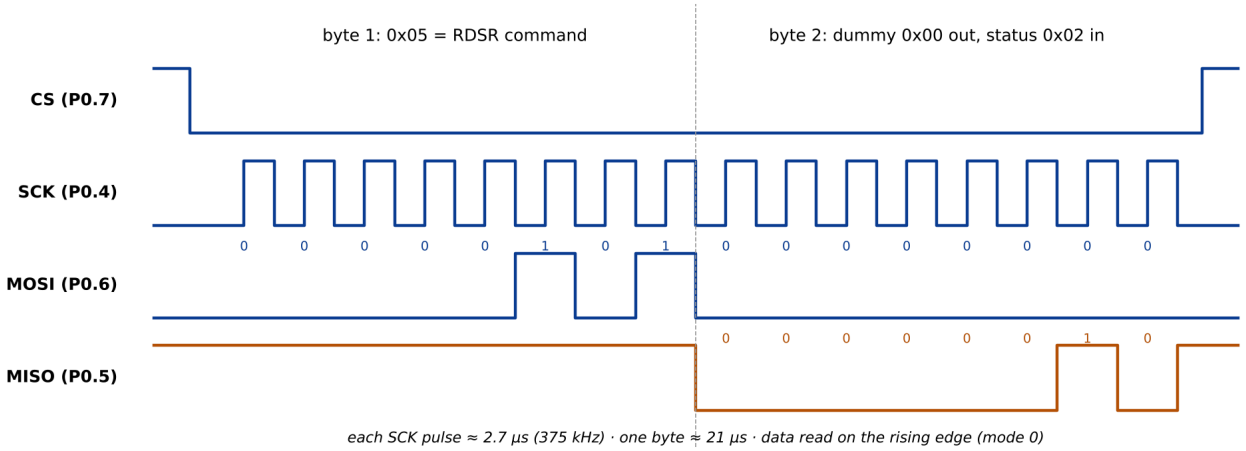
128	`    if (spi_err) return 0;` If an earlier SPI error happened, do nothing (no pins move).
129	`    S0SPDR = tx;` **Write the byte to `S0SPDR` → the SPI hardware starts:** it drives **8 clock pulses on SCK (P0.4 → EEPROM pin 6)** at 375 kHz, shifts the 8 bits out **MSB first on MOSI (P0.6 → EEPROM SI, pin 5)**, and at the same time **reads 8 bits from MISO (P0.5 ← EEPROM SO, pin 2)**.

130–131	`    do {`
	`        s = S0SPSR;`
	Read the SPI status register.

132	`        if (s & 0x18) spi_err = 2;  /* mode fault / abort */` Bits 3/4 set → the SPI hardware reported an abort or a mode fault → error 2.
133	`        if (--timeout == 0) spi_err = 1;  /* no response */` Waited too long (the clock never finished) → error 1.

Line	Code → what happens in the hardware
134	`    } while ((s & 0x80) == 0 && spi_err == 0);` Repeat until bit 7 (**SPIF**) is 1 — the hardware sets it after the 8th clock, about **21 µs** after line 129.

135–136	`    return S0SPDR;                                  /* reading it clears the done flag */`
	`}`
	Read `S0SPDR`: the **byte that came in on MISO**. This read also clears SPIF, ready for the next byte.



EEPROM commands (lines 137–163)

Every EEPROM command looks the same on the wires: **CS goes low → opcode → (address) → data → CS goes high.** The EEPROM ignores SCK while CS is high.

Line	Code → what happens in the hardware

137–138	`unsigned char ee_status(void) {`
	`    unsigned char s;`
	`ee_status()` — read the EEPROM status register.

139	`    IO0CLR = CS; spi(0x05); s = spi(0); IO0SET = CS;      /* RDSR */` **CS (P0.7) → 0 V**: the EEPROM starts listening. Opcode **0x05 (RDSR)** goes out on MOSI. A dummy 0x00 is sent while the **EEPROM drives its status byte on SO**. **CS → 3.3 V**: command ends.

140–141	`    return s;`
	`}`
	Return the status: bit 0 = busy writing (WIP), bit 1 = write allowed (WEL), bits 2–3 = protection.

142–143	`void ee_read_page(unsigned int addr) {                  /* 64 bytes into buf[] */`
	`    int i;`
	`ee_read_page(addr)` — read 64 bytes.

144	`    IO0CLR = CS; spi(0x03); spi(addr >> 8); spi(addr & 0xFF);` **CS low**, opcode **0x03 (READ)**, then the address high byte and low byte.
145	`    for (i = 0; i < 64; i++) buf[i] = spi(0);` 64 dummy bytes go out; for each one the **EEPROM sends the next stored byte on SO** (it moves to the next address by itself). Stored into `buf[]`.

146–147	`    IO0SET = CS;`
	`}`
	CS high — the read ends. Whole page ≈ 67 bytes × 21 µs ≈ 1.4 ms.

148–149	`unsigned int ee_write_page(unsigned int addr, int n) {`
	`    int i; unsigned char s;`
	`ee_write_page(addr, n)` — write `n` bytes into one page.

Line	Code → what happens in the hardware
150	`    IO0CLR = CS; spi(0x06); IO0SET = CS;                    /* WREN */` **CS low, opcode 0x06 (WREN), CS high**: the EEPROM sets its internal **write-enable latch (WEL)**. Without it the chip ignores writes.

151–153	`    s = ee_status();`
	`    if (spi_err) return spi_err;`
	`    if ((s & 0x02) == 0) return 5;                    /* write enable not set */`
	Read the status; if the chip did not set WEL (bit 1) → error 5.

154	`    IO0CLR = CS; spi(0x02); spi(addr >> 8); spi(addr & 0xFF);` **CS low**, opcode **0x02 (WRITE)**, address high, address low.
155	`    for (i = 0; i < n; i++) spi(buf[i]);` `n` bytes from `buf[]` are clocked into the EEPROM's **64-byte page buffer**.
156	`    IO0SET = CS;` **CS high → the EEPROM starts programming** its memory cells internally (up to 5 ms). The SPI bus is free.

157–161	`    for (i = 0; i < 20000; i++) {                    /* wait while write in progress */`
	`        s = ee_status();`
	`        if (spi_err) return spi_err;`
	`        if ((s & 0x01) == 0) return 0;`
	`    }`
	Read the status again and again: **bit 0 (WIP) = 1 while the chip is still writing**. When it becomes 0 the data is stored permanently.

162–163	`    return 6;`
	`}`
	Still busy after 20 000 checks (≈ 1 s) → error 6.

Profile pages: checksum, encode, decode (lines 165–198)

This part only works on RAM (`buf[]` and the profile arrays). **No pins change here** — the SPI functions above move the data in and out of the EEPROM.

Line	Code → what happens in the hardware
165	`/* ---------- Profile page: 'M' 'P' 3 id steps repeats name[6], 6 bytes/step, checksum ---------- */` Comment: page layout — 'M' 'P' 3 id steps repeats name, 6 bytes per step, checksum.

166–170	`unsigned char checksum(int n) {                    /* sum of buf[0..n-1] */`
	`    unsigned char s = 0; int i;`
	`    for (i = 0; i < n; i++) s += buf[i];`
	`    return s;`
	`}`
	`checksum(n)` : adds the first `n` bytes of `buf[]` in an 8-bit variable (the result wraps at 256).

171–172	`int encode(int k) {                                /* default profile k into buf[] */`
	`    int i, n = 12;`
	`encode(k)` : build default profile `k` in `buf[]` ; `n` = position after the 12-byte header.

173–174	`    buf[0] = 'M'; buf[1] = 'P'; buf[2] = 3; buf[3] = k; buf[4] = def_steps[k]; buf[5] = def_reps[k];`
	`    for (i = 0; i < 6; i++) buf[6 + i] = def_name[k][i];`
	Header: 'M', 'P', version 3, profile number, steps, repeats, then the 6-letter name.

Line	Code → what happens in the hardware
175–180	`for (i = 0; i < (int)def_steps[k]; i++, n += 6) { buf[n] = def_dir[k][i]; buf[n + 1] = def_speed[k][i]; buf[n + 2] = def_ramp[k][i] >> 8; buf[n + 3] = def_ramp[k][i] & 0xFF; buf[n + 4] = def_run[k][i] >> 8; buf[n + 5] = def_run[k][i] & 0xFF; }` 6 bytes per step: direction, speed, ramp time (high byte, low byte), run time (high byte, low byte). `>> 8` takes the high byte, `& 0xFF` the low byte.
181–183	`buf[n] = 0 - checksum(n); /* whole record now sums to 0 */ return n + 1; }` Checksum byte = 0 − sum, so **all bytes add up to 0**. Returns the length (DEMO: 37 bytes).
184–185	`int decode(int k) { /* buf[] into profile k; 1 if valid */ int i, n = buf[4];` `decode(k)` : check `buf[]` and copy it into the profile arrays.
186	`if (buf[0] != 'M' || buf[1] != 'P' || buf[2] != 3 || buf[3] != k) return 0;` Must start 'M','P', version 3 and have the right profile number — an empty chip (all 0xFF) fails here.
187	`if (n < 1 || n > 8 || buf[5] < 1 || buf[5] > 9 || checksum(13 + 6 * n) != 0) return 0;` 1–8 steps, 1–9 repeats, and the checksum over the whole record must be 0 — any changed byte fails.
188	`for (i = 0; i < n; i++) if (buf[12 + 6 * i] > REV || buf[13 + 6 * i] > 100) return 0;` Every direction must be ≤ 2 and every speed ≤ 100 %.
189–194	`for (i = 0; i < n; i++) { dir[k][i] = buf[12 + 6 * i]; speed[k][i] = buf[13 + 6 * i]; ramp[k][i] = buf[14 + 6 * i] * 256 + buf[15 + 6 * i]; run[k][i] = buf[16 + 6 * i] * 256 + buf[17 + 6 * i]; }` Copy the steps into RAM; `high × 256 + low` rebuilds each 16-bit time.
195–198	`for (i = 0; i < 6; i++) name[k][i] = buf[6 + i]; name[k][6] = 0; nsteps[k] = n; reps[k] = buf[5]; return 1; }` Copy the name, end it with 0, store the counts; return 1 = valid.

Loading the profiles (lines 199–215)

Line	Code → what happens in the hardware
199–200	`unsigned int load_profiles(void) { /* bad page: write default, read back */ int k; unsigned int e;` `load_profiles()` — runs once at start-up.
201–202	`unsigned char s = ee_status(); if (spi_err) return spi_err;` **SPI: RDSR** (CS low, 0x05, dummy, CS high).
203	`if (s == 0xFF) return 3; /* no EEPROM */` Status **0xFF** means MISO stayed at 3.3 V for all 8 bits — **no EEPROM answered** (missing or unpowered) → error 3.
204	`if ((s & 0x0C) == 0x0C) return 4; /* write-protected */` Both protection bits set → the EEPROM is **write-protected** → error 4.
205–206	`for (k = 0; k < 6; k++) { ee_read_page(k * 64);` For each of the 6 pages: **SPI READ** of 64 bytes from address `k × 64` (0, 64, 128, …).

Line	Code → what happens in the hardware

207–208	`if (spi_err) return spi_err;`
	`if (decode(k)) continue;`
	SPI error → stop. Page valid → go to the next page (nothing is written).

209	`if ((e = ee_write_page(k * 64, encode(k))) != 0) return e;` Page empty or damaged → build the default in `buf[]` and **SPI WRITE it to the EEPROM** (WREN, WRITE, wait ≈ 5 ms).

210–212	`ee_read_page(k * 64);`
	`if (spi_err) return spi_err;`
	`if (!decode(k)) return 9;`
	SPI READ the page back and check again; still wrong → error 9.

213–215	`}`
	`return 0;`
	`}`
	All six profiles are in RAM → return 0. Total time ≈ 9 ms (≈ 40 ms on a blank chip).

Motor control (lines 217–263)

Line	Code → what happens in the hardware

217–218	`/* ---------- Motor ---------- */`
	`void set_speed(unsigned int p) { duty = p; service(); }`
	`set_speed(p)` : set the PWM duty and call `service()` so **EN1 changes at once**.

219–220	`void set_dir(unsigned int d) {  /* IN1 and IN2 never high together */`
	`IO0CLR = IN1 | IN2;`
	`set_dir(d)` : first **IN1 (P0.12) and IN2 (P0.13) both go to 0 V**. They are never high together.

221	`if (d == FWD) IO0SET = IN1;` Forward: **IN1 → 3.3 V**. While EN1 is high, L293D **output 1 (pin 3) goes to 12 V** and output 2 (pin 6) to 0 V → current flows through the motor one way.
222	`if (d == REV) IO0SET = IN2;` Reverse: **IN2 → 3.3 V** → output 2 at 12 V, output 1 at 0 V → current flows the other way → **motor turns backwards**.
223	`if (d == STOP && cur_dir != STOP) { last_dir = cur_dir; off_since = now; }` When switching off, remember the old direction and the time (RAM) — needed for the 200 ms pause. At this moment duty is 0, so EN1 is low and the L293D outputs are off: **the motor coasts**.

224–225	`cur_dir = d;`
	`}`
	Store the new direction.

226–229	`int ramp_to(unsigned int target, unsigned int ms) {  /* 1 % steps, exactly ms long */`
	`unsigned int from = duty, start = now, n, k;`
	`n = (target > from) ? target - from : from - target;`
	`if (n == 0) return 1;`
	`ramp_to(target, ms)` : number of 1 % steps between the present and the new speed; nothing to do if equal.

230–233	`for (k = 1; k <= n; k++) {`
	`if (!wait_until(start + ms * MS * k / n)) return 0;`
	`set_speed(target > from ? from + k : from - k);`
	`}`
	Every step, at a precisely planned time, **the EN1 on-time grows or shrinks by 1 % (0.1 ms)** → the average motor voltage changes smoothly → the motor speeds up or slows down gently instead of jumping.

Line	Code → what happens in the hardware

234–236	`    now = start + ms * MS;`
	`    return 1;`
	`}`
	The ramp ends exactly `ms` after it started.

237	`void set_phase(unsigned int ph, unsigned int target) { phase = ph; show_status(target); }` `set_phase` : store the phase (the **LED pattern changes**) and prepare the new **LCD screen**.

238–239	`void run_step(void) {`
	`    unsigned int d = dir[profile][step];`
	`run_step()` : the direction this step wants.

240–244	`    if (cur_dir != STOP && d != cur_dir) {               /* slow down before changing direction */`
	`        set_phase(DECEL, 0);`
	`        if (!ramp_to(0, ramp[profile][step])) return;`
	`        set_dir(STOP);`
	`    }`
	Motor turning a different way (or the step is STOP): LCD “DECEL”, **EN1 pulses shrink to 0**, then **IN1 and IN2 → 0 V**.

245–250	`    if (d != STOP && d != cur_dir) {`
	`        if (last_dir != STOP && last_dir != d && (int)(off_since + 200 * MS - now) > 0) {`
	`            set_phase(DEAD, 0);                          /* 200 ms off before reversing */`
	`            now = off_since + 200 * MS;`
	`            if (!wait_until(now)) return;`
	`        }`
	Reversing less than 200 ms after switching off: LCD “DEAD”, **all outputs stay off** until 200 ms have passed — protects the motor and driver from a current spike.

251–252	`        set_dir(d);`
	`    }`
	Set IN1 or IN2 high for the new direction.

253–256	`    if (d != STOP) {`
	`        set_phase(ACCEL, speed[profile][step]);`
	`        if (!ramp_to(speed[profile][step], ramp[profile][step])) return;`
	`    }`
	LCD “ACCEL”, **EN1 pulses widen** step by step to the target speed.

257–259	`    set_phase(HOLD, duty);`
	`    now += run[profile][step] * MS; wait_until(now);`
	`}`
	LCD “HOLD”; **EN1 stays at the target duty** for the run time (the next planned time is `now + run`).

260–263	`void run_profile(void) {`
	`    for (rep = 0; rep < reps[profile]; rep++)`
	`        for (step = 0; step < nsteps[profile]; step++) { run_step(); if (skip) return; }`
	`}`
	`run_profile()` : runs all steps, repeated `reps` times; stops early if NEXT was pressed.

main() — start-up (lines 265–283)

Line	Code → what happens in the hardware

265–267	`/* ---------- Main ---------- */`
	`int main(void) {`
	`    int i;`
	Section comment, start of `main()` , a loop counter.

268	`    PLLCON = 0; PLLFEED = 0xAA; PLLFEED = 0x55;          /* PLL off: CPU at 12 MHz */` **PLL switched off and disconnected** (the 0xAA, 0x55 “feed” makes it take effect) → the **CPU now runs directly from the 12 MHz crystal**.

Line	Code → what happens in the hardware
269	`VPBDIV = 0; SCS = 0;  /* peripherals at 3 MHz */` **Peripheral clock = 12 MHz ÷ 4 = 3 MHz**; ports use the legacy GPIO registers.
270	`PINSEL0 = (PINSEL0 & 0x30FC00F3) | 0x1500;  /* P0.4-P0.6 = SPI, rest GPIO */` **Pin multiplexers:** P0.4, P0.5, P0.6 are connected to the **SPI block** (SCK, MISO, MOSI); P0.1, P0.7, P0.8, P0.12, P0.13, P0.15 to GPIO.
271	`PINSEL2 = PINSEL2 & ~8;  /* P1.16-P1.25 = GPIO */` P1.16–P1.25 are disconnected from the trace port → GPIO for the LCD.
272	`IO0SET = CS; IO0CLR = LED | EN1 | IN1 | IN2; IO0DIR |= LED | CS | EN1 | IN1 | IN2;` Output values are set first — **CS high, LED/EN1/IN1/IN2 low** — then the **output drivers are switched on**. At this moment the EEPROM is deselected, the **motor is off**, the LED is dark. Setting levels before enabling outputs avoids glitches.
273	`IO1CLR = 0x3F << 16; IO1DIR |= 0x3F << 16;` **LCD pins → 0 V**, then made outputs.
274	`PCONP |= (1 << 2) | (1 << 8);  /* Timer1 and SPI0 on */` **Clocks of Timer1 and SPI0 switched on.**
275	`T1TCR = 2; T1PR = FOSC / 4 / 100000 - 1; T1TCR = 1;  /* Timer1 tick = 10 us */` Timer1 **reset to 0**, prescaler 29 (counts every 30 × 1/3 MHz = **10 µs**), then **started**.
276	`pwm_start = ms_mark = T1TC;` Start the PWM period and millisecond counter at the present time.
277	`S0SPCCR = 8; S0SPCR = 0x20;  /* SPI master, mode 0, 375 kHz */` **SPI clock = 3 MHz ÷ 8 = 375 kHz**; SPI becomes **master, mode 0** — SCK rests at 0 V.
278	`delay_ms(40);  /* LCD set-up: 3, 3, 3, 2, settings */` Wait **40 ms** so the LCD finishes its own power-up reset.
279	`for (i = 0; i < 4; i++) { lcd_nibble(0, i < 3 ? 3 : 2); delay_ms(i == 0 ? 5 : 1); }` Four E pulses with 3, 3, 3, 2 on D4–D7 → the LCD switches to the **4-bit interface**.

280–281	`lcd_byte(0, 0x28); delay_ms(1); lcd_byte(0, 0x0C); delay_ms(1);`
	`lcd_byte(0, 0x06); delay_ms(1); lcd_byte(0, 0x01); delay_ms(2);`
	Commands: 0x28 two lines, 0x0C display on without cursor, 0x06 cursor moves right, 0x01 **clear the screen**.

282	`lcd_clear(); lcd_text(0, "MOTOR PROFILES"); lcd_text(16, "Loading EEPROM"); lcd_left = 34;` Start-up text queued — **“MOTOR PROFILES / Loading EEPROM”** appears during the next waits.
283	`for (i = 0; i < 3; i++) { pin(LED, 1); delay_ms(100); pin(LED, 0); delay_ms(100); }` **P0.1 high 100 ms, low 100 ms, three times → the LED blinks 3 times.**

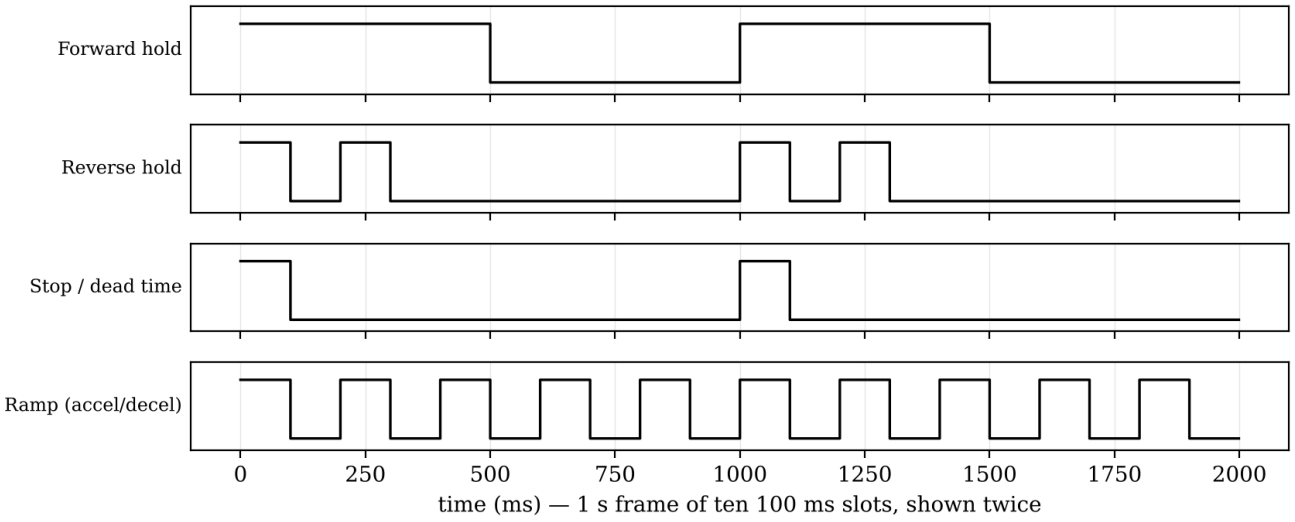
main() — loading and the main loop (lines 284–299)

Line	Code → what happens in the hardware
284	`fail = load_profiles();` **SPI traffic to the EEPROM**: status, then six page reads (and writes on a new chip).

Line	Code → what happens in the hardware
285–289	`if (fail) { /* EEPROM fault: LED on, motor off */ lcd_clear(); lcd_text(0, "EEPROM ERROR"); lcd_text(16, "Fail code"); lcd_num(26, 1, fail); lcd_left = 34; pin(LED, 1); while (1) service(); }` If loading failed: LCD **“EEPROM ERROR / Fail code n”**, **LED pin held at 3.3 V (on)**, motor pins stay at 0 V forever (`while (1) service();` only keeps the LCD going).
290	`btn = (IO0PIN >> 15) & 1; running = 1; now = T1TC; /* start the profiles */` Read the **button pin** once; enable LED patterns and the button; **start the schedule** at the present timer value.
291–292	`while (1) { if (!skip) run_profile();` Forever: run the current profile — the motor follows its steps.
293–296	`if (skip) { /* NEXT: ramp down, stop, next profile */ skip = 0; now = T1TC; set_phase(DECEL, 0); ramp_to(0, 300); set_speed(0); now = T1TC; set_dir(STOP); }` NEXT pressed: LCD “DECEL”, **EN1 ramps to 0 in 300 ms**, duty forced to 0, **IN1/IN2 → 0 V** — motor stopped.
297–299	`if (++profile == 6) { profile = 0; cycles++; } } }` Next profile; after MIXER go back to DEMO and count one full cycle.

3. What happens on the pins after switch-on

Time	Code lines	What you would see on the hardware
0 ms	268–277	CPU at 12 MHz; CS pin high; EN1, IN1, IN2, LED low (motor off); Timer1 starts counting; SPI ready
0–55 ms	278–281	LCD pins pulse: wake-up nibbles and set-up commands; the LCD screen is cleared
55–655 ms	282–283	LED blinks 3 times; “MOTOR PROFILES / Loading EEPROM” appears on the LCD
≈ 0.66 s	284	Bursts on CS, SCK, MOSI, MISO: the EEPROM is read (≈ 9 ms; ≈ 40 ms on a new chip because the six pages are written)
0.7 s	290–292	DEMO starts: IN1 goes high; EN1 starts pulsing every 10 ms with growing width; the motor speeds up; LED blinks fast
1.2 s	257–258	EN1 pulses steady at 7.5 ms of every 10 ms; motor at constant speed; LED slow blink
4.2 s	240–243	EN1 pulses shrink to zero, then IN1 drops: motor coasts; LED single flash
5.5 s	251–255	IN2 goes high, EN1 pulses grow to 4 ms of 10 ms: motor runs in reverse; LED double flash
10.3 s … 88.9 s	291–297	WASH, SPIN, GATE, FAN, MIXER follow; the sequence then repeats


Forward hold
Reverse hold
Stop / dead time
Ramp (accel/decel)
0
250
500
750
1000
1250
1500
1750
2000
time (ms) — 1 s frame of ten 100 ms slots, shown twice

4. Test yourself — “what does this line do to the hardware?”

Line	Your answer should mention
268	PLL off and disconnected; CPU clock = 12 MHz crystal; feed sequence makes it take effect
270	Pin multiplexers: P0.4/5/6 connected to SPI; other pins GPIO
272	CS pin high, motor and LED pins low, then outputs enabled — no glitch
275	Timer1 reset, prescaler 29 → counts every 10 µs, started
277	SCK = 375 kHz, master, mode 0
74–76	RS and data bits on the LCD pins, E pulse high then low → LCD reads 4 bits
91	Reads the voltage on P0.15: 1 released, 0 pressed
96	EN1 high for duty % of every 10 ms → average motor voltage ≈ duty × 12 V
100	LED on/off according to the pattern
129	SPI sends 8 bits on MOSI with 8 SCK pulses and receives 8 bits on MISO
139	CS low, RDSR, status byte comes back on MISO, CS high
150	CS low, WREN, CS high → EEPROM write-enable latch set
156	CS high after WRITE → EEPROM starts its internal write (≤ 5 ms)
203	Status 0xFF = MISO stuck high = no EEPROM answering
220–222	Both IN pins low first, then one high → motor direction
230–233	EN1 pulse width changes 0.1 ms per step → smooth speed change
245–250	Outputs kept off 200 ms before reversing
285–289	Fault: LED held on, motor pins stay low, LCD shows the code

> ***One-line summary for the viva:*** *“Register writes change pins immediately:* `IO0SET/IO0CLR` *drive the LED, CS, EN1, IN1, IN2; writing* `S0SPDR` *makes the SPI block clock a byte to the EEPROM; the LCD reads its pins when E falls; and Timer1, counting every 10 µs, decides when each of these happens.”*